

SOAL 3 — Troubleshooting & Root Cause Analysis

Nama : Hafizh 'Abid Khalish

Tanggal : 17-09-2026

Sistem yang Dianalisis : bridge controller yang membaca data dari device lewat serial lalu mengirimnya ke server.

Tahap 1. Memahami Gejala yang Terjadi dari Penyebab yang Diketahui

Kode	Gejala	Yang bisa disimpulkan
G1	Data kadang masuk kadang tidak	Kehilangan data sesekali, bukan total
G2	Sebagian data undefined/null	Data sampai tapi isinya rusak ini masalah integritas kesesuaian, bukan koneksi
G3	API kadang terima kadang gagal	Kegagalan ada di sisi pengiriman
G4	Berhenti terima data setelah beberapa jam	Memburuk bertahap, bukan mati mendadak
G5	Restart aplikasi bikin normal sementara	Ada sesuatu yang menumpuk dan hilang saat proses direstart

Diketahui pada G5, restart proses hanya mereset apa yang dipegang proses itu sendiri yakni memori, socket, timer, dan event listener. Restart tidak membenahi konektor longgar, tidak mengubah baud rate firmware, dan tidak menambah kapasitas server. Karena itu penyebab G4 hampir pasti ada di lapisan aplikasi, berupa resource yang menumpuk dan tidak pernah dilepas. Pola waktu G2 (muncul acak sejak awal) berbeda dari G4 (bertahap setelah berjam-jam), sehingga keduanya diperlakukan sebagai dua masalah terpisah yang kemungkinan saling memperburuk: tiap data rusak atau koneksi putus memicu jalur error, dan kalau jalur error itu sendiri yang bocor, sistem makin cepat habis.

Tahap 2. Mencari Kemungkinan Penyebab dari Setiap Lapisan Sistem

Lapisan	Kemungkinan penyebab	Menjelaskan gejala
Hardware	Konektor/kabel serial longgar	G1, G2
	Power supply drop saat beban berubah	G1, G4
	Gangguan elektromagnetik dari motor/kontaktor	G2
Serial	Data diparse tanpa menunggu newline (framing)	G2
	Baud rate device dan aplikasi berbeda	G2
	Buffer penerimaan penuh, baca lebih lambat dari kirim	G1, G4
	Nama port berubah setelah reconnect	G4
Aplikasi	Event listener menumpuk tiap reconnect	G1, G3, G4, G5
	Antrian/buffer retry tanpa batas ukuran	G4, G5
	Socket tidak dilepas karena tidak ada timeout	G3, G4, G5
	Timer reconnect bertumpuk (error + close bersamaan)	G4, G5
API/Network	Request tanpa timeout, menggantung selamanya	G3, G4
	Connection pool habis, koneksi tak dikembalikan	G3, G4, G5
	Server membatasi jumlah request per waktu	G3

Tahap 3. Membuat Urutan untuk Menyelesaikan Masalah

1. Membaca log yang sudah ada untuk mencari kapan error pertama muncul dan apakah jaraknya makin pendek. Nol risiko, tidak merusak bukti.
2. Memasang pemantauan resource pada proses (memori, file descriptor, socket, jumlah listener). Bisa membuktikan atau menggugurkan dugaan akumulasi tanpa mengubah kode.
3. Merekam data serial mentah sebelum diparse untuk memisahkan G2 (framing aplikasi) dari kemungkinan penyebab fisik/ baud rate.
4. Periksa jalur fisik (konektor, kabel, ground, daya device). Ditunda sampai di sini karena G5 sudah mengarahkan dugaan ke aplikasi, dan pemeriksaan fisik butuh akses lapangan.
5. Bandingkan jumlah request yang dikirim klien dengan yang tercatat di server, untuk memastikan di sisi mana data sebenarnya hilang.
6. Reproduksi dipercepat, putuskan koneksi device tiap 30 detik, amati apakah listener dan memori naik mengikuti jumlah reconnect.
7. Terapkan perbaikan satu per satu, tiap perbaikan diikuti pengamatan.

Tahap 4. Melakukan Hipotesis untuk Menemukan Akar Masalah

Untuk G1, G3, G4, G5

Resource pada lapisan aplikasi menumpuk dan tidak pernah dilepas, dipicu siklus reconnect yang tidak membersihkan koneksi lama terutama event listener yang menumpuk, diperparah antrian dan socket yang tidak dibatasi.

- Satu-satunya dugaan yang menjelaskan G4 dan G5 sekaligus yakni sistem yang memburuk pelan lalu normal lagi setelah restart adalah ciri khas state yang menumpuk di dalam proses.
- Cocok dengan sifat G1/G3 yang kadang muncul: di awal tumpukan masih sedikit, setelah listener berlipat, satu data diproses berkali-kali dan socket cepat habis.
- Pemicu siklusnya sudah tersedia dari gangguan serial G1/G2 tiap gangguan memicu reconnect, tiap reconnect menambah kebocoran.
- Dugaan hardware murni atau server kelebihan beban gugur oleh G5, karena keduanya tidak akan sembuh hanya karena proses klien direstart.

Untuk G2

Aplikasi mem-parse tiap potongan data serial apa adanya tanpa menunggu newline. Port serial mengirim stream byte, bukan pesan utuh pada satu event data bisa berisi separuh JSON, sehingga sebagian field hilang menjadi undefined. Kemungkinan lain (baud rate berbeda, gangguan fisik) dibedakan lewat data mentah pada Langkah 3.

Tahap 5. Membuat Usulan untuk Memperbaiki Masalah yang Ditemukan

Kode	Perbaikan	Menangani
P1	Lepas listener lama sebelum membuang koneksi	Listener menumpuk
P2	Satu reconnect terjadwal (guard timer)	Timer bertumpuk
P3	Potong stream serial berdasar newline	Framing rusak (G2)

P4	Timeout di tiap request API	Socket menggantung
P5	Jangan ulang request yang dibalas 4xx	Beban server sia-sia
P6	Batasi ukuran antrian/buffer retry	Antrian tanpa batas
P7	Pasang metrik resource periodik	Kebocoran baru ketahuan saat mati

Tahap 6. Verifikasi Hasil dari Perbaikan Masalah yang Telah Dilakukan

Yang dipantau	Target	Membuktikan
Pemakaian memori (RSS)	Datar setelah 30 menit, naik < 5% / 48 jam	Kebocoran resource
Jumlah listener	Tetap, meski reconnect sudah ratusan kali	Listener leak (utama)
Jumlah socket ke API	Stabil, kembali normal saat idle	Connection pool
Data tidak valid	Tidak bertambah seiring waktu	Masalah framing
Jumlah restart dibutuhkan	Nol dalam 48 jam	G5 teratasi

Uji cepat dengan paksa 500 siklus reconnect dalam waktu singkat, lalu bandingkan jumlah listener dan memori di awal-akhir. Kalau keduanya tetap datar, root cause A1 terkonfirmasi. Kalau masih naik, ambil heap snapshot untuk menemukan objek yang tertahan. Perbaikan ini mengatasi akibat dari gangguan serial, bukan gangguan itu sendiri. Bila jumlah reconnect tetap tinggi setelah perbaikan, kemungkinan masih ada masalah fisik yang perlu ditangani terpisah oleh karena itu sistemnya kini hanya lebih tahan menghadapinya.